Semana 1 · Microcontroladores y arquitectura

Unidad 1 · CPU, memorias, buses, bits, periféricos y señales

1. Microcontrolador / sistema embebido

Microcontrolador -> circuito integrado que reúne procesamiento, memoria y periféricos. Se programa para cumplir una tarea concreta, no para reemplazar a una computadora completa.

↳ Yo lo entiendo como un pequeño computador dentro de un circuito. Recibe datos de los pines, los procesa según el programa y luego hace algo con las salidas... Por ejemplo, detectar un botón y prender una luz.

ESP32 -> CPU + RAM + periféricos + conectividad inalámbrica.

Sensor -> entrada -> ESP32 procesa -> salida (LED / alarma).

- Microprocesador -> principalmente realiza el procesamiento; necesita otros componentes para formar un sistema completo.
- Microcontrolador -> integra más elementos dentro del mismo chip, normalmente consume menos y controla equipos específicos.
- Sistema embebido -> dispositivo con una función definida: control de luces, acceso, alarma, lectura de sensores...

Nota al margen: En el ESP32 todo gira alrededor de leer una entrada, decidir y cambiar la salida...

2. CPU y arquitectura interna

CPU -> busca -> decodifica -> ejecuta instrucciones (ciclo repetido).

↳ La CPU trabaja siguiendo instrucciones. Primero busca la siguiente, interpreta qué tiene que hacer y después la ejecuta. Todo esto ocurre muchas veces por segundo.

- ALU -> operaciones aritméticas y lógicas: sumar, restar, AND, OR, XOR...
- Registros -> pequeñas memorias rápidas dentro del procesador; guardan datos de trabajo.
- Unidad de control -> coordina qué instrucción ejecutar y cuándo.
- Reloj -> marca el ritmo interno; frecuencia en Hz / MHz.
- Periféricos -> GPIO, temporizadores, interfaces de comunicación, ADC...

Buses (lo anotado en clase)

Bus de datos -> transporta información.

↳ En el dibujo de clase los buses conectan CPU, memoria y periféricos. El de datos mueve valores; el de direcciones dice a qué lugar se quiere acceder, y el de control indica la operación.

Bus de direcciones -> indica dónde leer / escribir.

Bus de control -> lectura, escritura, habilitaciones...

Von Neumann -> modelo donde programa y datos comparten el camino principal de memoria. Harvard -> separa los caminos de instrucciones y datos. Son modelos para entender cómo se organiza el acceso.

Idea del dibujo -> CPU ↔ memoria ↔ entradas y salidas.

3. Memorias: cuáles pierden datos y cuáles no

Volátil -> al quitar la alimentación se pierde la información. No volátil -> conserva lo almacenado.

↳ Aquí la diferencia clave es si la información sigue guardada al apagar el equipo. La RAM sirve mientras el ESP32 está funcionando; la Flash conserva el programa cargado.

RAM -> datos mientras corre el programa (variables, cálculos).

SRAM -> RAM estática; muy rápida y no necesita refresco periódico.

↳ SRAM → rápida, sin refresco. DRAM → usa capacitores y necesita refrescarse. No significa que el ESP32 tenga que usar todas estas memorias; son tipos que comparamos en clase.

DRAM -> RAM dinámica; guarda carga y necesita refresco.

Caché -> memoria muy rápida y pequeña cercana a la CPU; guarda información usada con frecuencia.

ROM -> memoria de solo lectura como concepto general.

PROM -> puede programarse una sola vez.

EPROM -> se puede borrar usando luz ultravioleta (tecnología antigua).

EEPROM -> se borra y escribe eléctricamente; no es igual a RAM.

Flash -> no volátil; normalmente conserva el firmware del ESP32.

- En el ESP32: el programa se almacena en Flash; datos de ejecución -> RAM. No confundir dónde se guarda el código con dónde trabajan las variables.
- SSD / disco -> almacenamiento secundario en computadoras; no es lo mismo que la memoria interna del microcontrolador.

Nota al margen: RAM = rápida y temporal; Flash = queda guardado al apagar...

4. Bits y conversiones (ejercicios de las hojas)

Bit -> 0 o 1. Byte -> 8 bits.

Decimal -> base 10. Binario -> base 2. Hexadecimal -> base 16 (0...9, A...F).

1 dígito hexadecimal = 4 bits; 0x20 = 32 decimal; 0xFF = 255.

32 decimal -> 0010 0000 binario. 10 decimal -> 0000 1010 binario.

Binario a decimal -> sumar potencias de 2 donde haya un 1. Decimal a binario -> dividir entre 2 y leer los residuos desde abajo.

Operaciones bit a bit

A	B	AND (&)	OR (|)	XOR (^)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

NOT (~) -> invierte bits. << -> desplaza a izquierda. >> -> desplaza a derecha.

Ejemplo: 1010 & 1100 = 1000; 1010 | 1100 = 1110.

Máscara de bits -> sirve para revisar, activar o limpiar bits específicos de un dato.

valor & mascara -> revisar bits elegidos; valor | mascara -> activar los que indique la máscara.

5. GPIO: entradas / salidas del ESP32

GPIO = General Purpose Input/Output. Pin programable para recibir o entregar señales digitales.

GPIO -> INPUT (botón, sensor digital) / OUTPUT (LED, buzzer).

↳ Un GPIO se configura según lo que quiero conectar. Si leo un pulsador es entrada; si controlo un LED es salida. Hay que comprobar los pines, porque algunos tienen restricciones.

HIGH -> nivel alto (~3,3 V en la salida típica). LOW -> cerca de 0 V.

GND -> tierra común del circuito; sin referencia común puede fallar la lectura.

- Si el botón deja la entrada sin nivel definido -> lectura inestable (pin flotante). Usar pull-up / pull-down cuando corresponda.
- No todos los GPIO sirven para todo: algunos están reservados o tienen funciones especiales según el módulo.
- Importante -> ESP32 no debe recibir 5 V directamente en un GPIO normal. Revisar el voltaje antes de conectar.

LED -> GPIO -> resistencia -> LED -> GND (según conexión).

Nota al margen: Verificar pin / polaridad del LED / GND antes de probar...

6. Periféricos que aparecen en el sílabo

UART -> comunicación serial con TX y RX; normalmente entre dos equipos.

↳ Son formas distintas de comunicar dispositivos. UART → TX/RX; I2C → dos líneas compartidas; SPI → reloj y líneas de datos. Anotación: revisar siempre cómo se conectan...

I2C -> SDA (datos) + SCL (reloj), varios dispositivos en el mismo bus.

SPI -> líneas de datos, reloj y selección de dispositivo; comunicación rápida.

ADC -> convierte un voltaje analógico en un número digital; lectura de sensor.

DAC -> convierte un número en una señal analógica de salida; depende del modelo de ESP32.

PWM -> serie de pulsos; modifica el porcentaje encendido (duty cycle).

Señal digital -> 0 / 1. Señal analógica -> puede tomar valores intermedios.

Ejemplo rápido: potenciómetro -> ADC -> número -> comparación -> cambiar salida. PWM -> brillo de LED o control de potencia, según el circuito.

Para cerrar la semana

CPU -> ejecuta. RAM -> trabaja. Flash -> guarda programa. GPIO -> conecta con el exterior.

Anotar después -> cuáles GPIO se usaron y el circuito que se armó...